

R1 Bimanual Neural Grasp, Handoff, and Throw

Two Revo 2 Basic hands controlled by teacher-guided residual learning

Cesar A. Pascal Vasquez

Manual 4

Purpose of this Manual

This manual completes the experimental progression by mounting right and left BrainCo Revo 2 Basic hands on the Unitree R1. One neural policy must reach and grasp a ball with the right hand, transport it to the left hand, complete the handoff, withdraw the right hand, and throw with the left arm. The procedure documents the teacher, network, training parameters, robustness methods, contact stabilizer, evaluation, and final playback commands.

Experiment goal

Reproduce and evaluate the saved 94-160-160-22 policy on the complete bimanual task using 100 held-out randomized MuJoCo episodes.

Contents

1	Bimanual R1 Model	3
2	Seven-Second Task Sequence	4
3	Observation Vector	5
4	How the Bimanual Teacher Works	6
5	Neural Residual Controller	7
6	Baseline Code Package and Data Flow	8
7	Bimanual Model-Builder Walkthrough	9
8	Environment, Teacher, and Contact Gate Code	10
9	Policy and Training Code Walkthrough	11
10	Initial Behavioral Cloning	12
11	Two DAgger Rounds	13
12	Domain Randomization	14
13	Contact-Gated Ball Stabilizer	15
14	Set Up, Train, and Run	16
15	Held-Out Results	17
16	Visual Result and Interpretation	18
17	Final Checklist and References	19

1 Bimanual R1 Model

This manual is independent and can be followed without the earlier experiments. You need Python 3.10 or newer, MuJoCo 3, NumPy, the Unitree R1 MJCF/meshes, both BrainCo Revo 2 XML/mesh folders, and the files in `editable_code/`. Nothing in this exercise controls physical hardware.

The model removes the original terminal hand geometry from both R1 wrists and mounts separate right and left Revo 2 Basic assets. Neutral sleeve-and-flange geometries visually join the vendor wrist meshes to the R1 forearms. Each hand retains six active controls.

The fixed-pelvis robot exposes 22 actions:

- five right-arm actuator commands;
- five left-arm actuator commands;
- six right-hand Revo 2 commands;
- six left-hand Revo 2 commands.

The integrated model is `model/r1_revo2_bimanual.xml`. Fixing the pelvis removes balance and locomotion from the experiment, allowing the study to focus on bimanual manipulation.

Required files and first model check

```
r1_bimanual_lab/  
|-- scripts/           # copy editable_code/scripts here  
|-- model/r1/          # R1 XML and meshes  
|-- model/revo2/       # right/left XML and meshes  
|-- policies/ results/  
  
python3 -m venv .venv  
.venv/bin/python -m pip install "mujoco>=3.3.6,<4" "numpy>=2,<3"  
.venv/bin/python scripts/build_model.py
```

The build must finish with 22 controlled actuators and create `model/r1_revo2_bimanual.xml`. Open that file and confirm that no stock terminal hand geometry overlaps either Revo 2 wrist.

Geometry assumption

The visual adapters are approximate and not manufacturing drawings. Contact behavior and reachable poses in this simulation must be revalidated against the physical wrist interface before deployment.

2 Seven-Second Task Sequence

The policy runs at 40 Hz for 7.10 s. The task is staged, but the network receives the full current state and can correct position errors within each phase.

Table 1: Nominal bimanual episode

Time	Required behavior
0.00–1.50 s	Right palm reaches the randomized ball
1.47 s	Begin checking valid right-hand contact
1.50–2.48 s	Right hand closes and secures the ball
2.05–3.82 s	Arms transport the ball toward the left receive pose
3.62–4.28 s	Left hand closes around the ball
4.32–4.58 s	Right hand opens and retreats after transfer
4.58–5.05 s	Left arm lifts the received ball
5.05–5.48 s	Left arm moves to the windup pose
5.75 s	Left hand releases the ball
5.75–6.06 s	Left arm follows through toward the target
6.06–7.10 s	Free flight and final metric calculation

A random zero-to-two-tick command delay changes the exact response time across episodes.

3 Observation Vector

The policy receives 94 normalized values at every control update. The observation includes information for both arms and both hands, as well as the previous 22-dimensional command.

Table 2: The 94-dimensional observation

Group	Size	Content
Task context	22	Normalized time and phase, ball relative to both palms, handoff and throw-target errors
Robot and object state	50	Palm and ball motion, both arm states, both hand ratios, contacts, fixture state
Action memory	22	Previous right-arm, left-arm, right-hand, and left-hand command
Total	94	Network input at each 40 Hz step

The phase encoding is important because identical ball positions can require different actions during transport, receive, or release. Previous-action memory helps the network compensate for the randomized command delay and encourages smooth corrections.

4 How the Bimanual Teacher Works

The teacher is a scripted controller built from task waypoints, inverse kinematics, and timed hand commands. It is not trained. It calculates a central solution for each randomized episode:

1. solve right-arm inverse kinematics for approach and grasp;
2. close the right hand and lift the ball;
3. move both palms toward a shared transfer envelope;
4. close the left hand while the right hand still supports the ball;
5. verify the receive condition, open the right hand, and retreat;
6. lift and wind up with the left arm;
7. open the left hand at release and follow through.

The demonstration set contains 140 randomized teacher episodes beginning at seed 24000. The scripted controller achieved 97.14% complete success over this collection. The teacher supplies the ordered skill; the neural network learns bounded corrections for the states produced by noise, delay, physical variation, and its own prediction errors.

Expected result

During DAgger, the teacher also labels states visited by the neural policy. This converts mistakes such as a late receive pose into supervised recovery examples.

5 Neural Residual Controller

The multilayer perceptron has 94 inputs, two 160-unit tanh hidden layers, and 22 tanh outputs. Its parameter count is

$$(94 \times 160 + 160) + (160 \times 160 + 160) + (160 \times 22 + 22) = 44,502.$$

The output is a residual around the teacher action. Arm residuals are limited to ± 0.24 and hand residuals to ± 0.10 before the final actuator limits are applied.

```
1 features = normalize(observation)
2 h1 = np.tanh(features @ W1 + b1)
3 h2 = np.tanh(h1 @ W2 + b2)
4 residual = np.tanh(h2 @ W3 + b3)
5
6 scale = np.r_[np.full(10, 0.24),
7               np.full(12, 0.10)]
8 action = np.clip(teacher + scale * residual, -1, 1)
```

Listing 1: Conceptual residual action

The policy file `policies/r1_dual_revo2_handoff_throw.npz` stores weights and observation-normalization statistics. Playback performs a small matrix computation at every 0.025 s control interval.

To write a different controller, preserve the same interface: a callable receives a 94-value NumPy array and returns 22 finite values in $[-1, 1]$. Begin with residual limits no larger than the reference values. The file `editable_code/scripts/bimanual_policy.py` is deliberately editable and contains the complete forward pass, target encoding, and save/load methods.

Network design choice

Choose hidden width 80–256 and one activation (tanh recommended for the first run). Predict whether the change affects training time, success rate, or both.

Expected effect: _____

6 Baseline Code Package and Data Flow

The full baseline source is supplied in `editable_code/`. Copy `requirements.txt` to the project root and `editable_code/scripts/` to the local `scripts/` folder. Preserve an unchanged copy before editing.

Table 3: Bimanual baseline files

File	Responsibility
<code>build_model.py</code>	Removes both stock terminals, mounts right/left Revo 2 assets, adds scene and actuators
<code>bimanual_env.py</code>	Task timing, 94 observations, teacher, randomization, contact gate, reward, and metrics
<code>bimanual_policy.py</code>	94-hidden-hidden-22 residual MLP, Adam, normalization, and NPZ I/O
<code>train_policy.py</code>	Teacher collection, cloning, DAgger, evaluation, CSV, JSON, and policy saving
<code>play_policy.py</code>	Interactive trained rollout
<code>evaluate_policy.py</code>	Held-out aggregate and per-episode JSON
<code>render_policy.py</code>	Trained, untrained, or teacher video

One control step follows this path: **environment observation** $\rightarrow$ **policy normalization** $\rightarrow$ **two tanh layers** $\rightarrow$ **residual decode** $\rightarrow$ **22 actuator commands** $\rightarrow$ **MuJoCo step** $\rightarrow$ **event/contact update**. During teacher collection, `teacher_action()` replaces or mixes with the neural command.

Model and policy versions must match

The saved network assumes 94 observations, 22 actions, and the exact actuator ordering in the assembled XML. If the model builder, observation order, or actuator list changes, record a new version and retrain.

7 Bimanual Model-Builder Walkthrough

Open `build_model.py`. Verify the constants `R1_SOURCE`, `RIGHT_SOURCE`, `LEFT_SOURCE`, and `OUTPUT`. The remainder of the file is a readable XML transformation pipeline.

Table 4: Where assembly behavior is implemented

Search for	What can be inspected or changed
<code>rewrite_mesh_paths</code>	Relative mesh prefixes after assets are copied
<code>joint_gains</code>	R1 actuator gains, damping, and force limits
<code>remove_stock_hand</code>	Names and descendants removed from each wrist
<code>add_adapter</code>	Approximate flange size, pose, material, and collision setting
<code>mount_hand</code>	Wrist body, hand base, name prefixes, and transform
<code>add_scene</code>	Ball, floor, cameras, targets, and soft fixture
<code>build</code>	Fixed pelvis, asset merge, contacts, 22 actuators, and output write

The physics option created in `build()` uses 0.0025 s, `implicitfast`, 32 solver iterations, eight line-search iterations, and Earth gravity. If contact becomes unstable, test a smaller time step before altering reward or training.

Required check after every builder edit

```
.venv/bin/python scripts/build_model.py
.venv/bin/python -c "import mujoco; m=mujoco.MjModel.from_xml_path('model/r1_revo2_bimanual.xml');
print(m.nu, m.njnt)"
```

The first number must support the complete model and the policy must still resolve its expected 22 actuator names. Open the model visually and inspect both wrists for duplicated geometry.

8 Environment, Teacher, and Contact Gate Code

Open `bimanual_env.py`. `TaskConfig` contains the complete phase clock. The other locations below define the experiment rather than the neural network.

Table 5: Exact bimanual environment edit locations

Search for	Behavior or parameter
<code>TaskConfig</code>	40 Hz rate, 7.10 s duration, reach/close/handoff/release times, delay, noise
<code>_restore_and_randomize_model</code>	Gains 0.84–1.16, radius 0.0168–0.0192 m, mass 0.040–0.065 kg, friction 0.85–1.35
<code>_compute_waypoints</code>	Right reach, shared handoff, left lift, windup, and throw targets
<code>teacher_action</code>	Both arm trajectories and twelve hand commands
<code>_observation</code>	The 94 feature values and ordering
<code>_update_events</code>	Right grasp, left receive, and release-state flags
<code>_update_carrier_target</code>	Contact-gated soft attachment target and disable time
<code>_reward</code>	Phase-dependent dense learning score
<code>metrics</code>	Stage success flags and final throw measurements

To study handoff timing, change `left_close_start`, `left_close_end`, and `right_release_start` together while preserving an overlap interval. A physically meaningful transfer requires the left hand to close before the right hand opens.

Example controlled variable

Baseline overlap: $4.28 - 3.62 = 0.66$ s before left closure ends. New overlap or delay: _____ s. Keep ball distribution, network, seed blocks, and success thresholds unchanged.

9 Policy and Training Code Walkthrough

Open `bimanual_policy.py`. The constructor controls hidden size, initialization seed, and residual scales. `_forward()` is the MLP; `encode_targets()` converts teacher actions into residual labels; `nominal_actions()` generates the central bimanual trajectory; and `train_batch()` implements weighted loss, weight decay, clipping, and Adam.

Open `train_policy.py`. Search for the following symbols instead of relying on line numbers.

Table 6: Training parameter-to-code map

Parameter	Change at	Baseline behavior
Policy output	<code>--output</code>	Keeps each trained NPZ separate
Teacher data	<code>collect_teacher_data</code>	Labels every visited state with teacher action
Hidden/budget/LR	command-line flags	Values documented in Sections 10–11
Loss weights	<code>output_weights</code>	4.0 for ten arm, 1.6 for twelve hand channels
Phase balance	<code>phase_buckets</code>	Samples nine stages uniformly
Residual bounds	<code>BimanualNeuralPolicy.__init__</code>	0.24 arm and 0.10 hand
Weight decay	<code>train_batch</code>	1×10^{-6}
Teacher mixing	<code>dagger_probabilities</code>	0.45 then 0.18
Seed blocks	<code>first_seed</code> calls	24000 teacher, 34000 DAgger, 64000 training evaluation

To compare two policies, use different `--output` paths during training and pass each path to `evaluate_policy.py` with `--policy`. Use one unseen evaluation seed block for both.

Example baseline and edited architecture

```
.venv/bin/python scripts/train_policy.py \
  --hidden-size 160 --output policies/baseline_h160.npz
.venv/bin/python scripts/train_policy.py \
  --hidden-size 224 --output policies/edited_h224.npz
.venv/bin/python scripts/evaluate_policy.py \
  --policy policies/baseline_h160.npz \
  --episodes 100 --first-seed 84000 \
  --output results/baseline_h160_eval.json
.venv/bin/python scripts/evaluate_policy.py \
  --policy policies/edited_h224.npz \
  --episodes 100 --first-seed 84000 \
  --output results/edited_h224_eval.json
```

10 Initial Behavioral Cloning

The first stage trains the network on the 140 teacher rollouts. The loss is a weighted mean-squared action error. Each of the ten arm outputs uses weight 4.0, and each of the twelve hand outputs uses weight 1.6. These values emphasize the palm trajectories without ignoring closure and release timing.

The initial stage uses:

- 3,000 Adam optimizer updates;
- batch size 512;
- hidden size 160;
- learning rate 1.1×10^{-3} ;
- weight decay 1×10^{-6} ;
- gradient clipping to the interval $[-2, 2]$;
- nine-phase balanced mini-batches.

Phase balancing ensures that a short but critical event, such as right-hand release or left-hand throw release, is represented in every useful sequence of batches. Without balancing, the longer reach and transport segments would dominate the loss.

11 Two DAgger Rounds

Behavioral cloning can accumulate errors because the network eventually reaches states that are rare in clean demonstrations. DAgger corrects this distribution shift.

For each of two rounds, the current policy runs 40 randomized episodes. The teacher labels the states encountered during those rollouts. The expanded dataset is then optimized for 1,500 updates with a DAgger learning rate of 8.25×10^{-4} .

Teacher action mixing is reduced as the policy improves: 45% in the first DAgger collection and 18% in the second. The final training record contains 55,926 state-action samples and 6,000 total optimizer updates:

$$3000 + 1500 + 1500 = 6000.$$

Table 7: Student-selectable training budget

Parameter	Reproduction	Study range	Main tradeoff
Teacher episodes	140	40–300	Coverage versus collection time
Initial updates	3,000	500–8,000	Initial imitation fit
DAgger rounds	2	0–4	Recovery data versus total cost
Episodes/round	40	15–100	Visited-state diversity
Updates/round	1,500	500–4,000	Fit to aggregated data
Batch size	512	128–1,024	Gradient noise and memory
Learning rate	1.1×10^{-3}	3×10^{-4} – 2×10^{-3}	Stability versus speed

Interpretation

The final policy is still teacher-guided because its output is a bounded residual. DAgger makes that residual reliable in policy-visited states; it does not remove the scripted reference trajectory.

12 Domain Randomization

Robustness comes from training on a distribution rather than a single scene. Every rollout changes several variables while retaining a physically meaningful task.

Table 8: Bimanual randomization ranges

Variable	Range or variation
Ball and targets	Randomized start, handoff center, and throw target
Ball radius	16.8–19.2 mm
Ball mass	40–65 g
Contact friction	0.85–1.35 scale range
Actuator gains	84–116% of nominal
Command delay	0–2 control ticks
Observation noise	Gaussian noise with $\sigma = 0.009$
IK initialization	Randomized seeds and small waypoint variation

Randomization tests whether feedback can correct perturbations. It does not prove real-world transfer because simulation may omit compliance, backlash, latency, tactile sensing, and calibration errors.

13 Contact-Gated Ball Stabilizer

The simulation uses a contact-gated soft attachment to keep the training problem numerically stable with approximate adapters and five-degree-of-freedom arms. Its state changes only when position and contact conditions are satisfied.

1. The ball begins softly supported while the open right hand approaches.
2. After a valid right-hand grasp envelope and contacts, the ball follows the right hand.
3. During handoff, support switches to the left only after the receive envelope, left closure, and contacts are valid.
4. Before the throw, the attachment is disabled so the measured trajectory is free ball flight.

This mechanism is a modeling aid, not a learned tactile behavior. It prevents early numerical slips from destroying teacher data, but it can make grasp retention easier than on the physical robot. Report both the contact gate and the free-flight release in any result that uses this environment.

Avoid a false success

The complete-task success flag requires the ordered reach, right grasp, transfer, left hold, release, and forward-flight conditions. A ball supported only by the stabilizer must not count as a successful throw.

14 Set Up, Train, and Run

Run the one-time setup from the project folder. The launcher creates `.venv`, installs the packages, and builds or validates the integrated XML.

One-time setup

```
cd "/path/to/r1_bimanual_lab"
zsh "./Setup MuJoCo.command"
```

Train only when a new network is required. The script uses the full parameters in Sections 10 and 11. Use a unique `--output` path for every experiment so the included policy is preserved.

Full neural training

```
.venv/bin/python scripts/train_policy.py \
--output policies/student_bimanual_policy.npz
```

For a student-designed run, supply explicit flags so the result is reproducible:

Example custom budget

```
.venv/bin/python scripts/train_policy.py \
--hidden-size 128 --teacher-episodes 100 \
--initial-steps 2200 --dagger-rounds 2 \
--dagger-episodes 30 --dagger-steps 1000 \
--batch-size 256 --learning-rate 0.0008 \
--output policies/student_custom_budget.npz
```

The shortest command to open the saved final policy is:

Final result

```
cd "/path/to/r1_bimanual_lab"
zsh "./Run Final Result.command"
```

The equivalent direct command runs three episodes:

Direct playback

```
.venv/bin/python scripts/play_policy.py --episodes 3
```

Evaluate the saved policy without a viewer and render matched trained and untrained videos.

Evaluation and video rendering

```
.venv/bin/python scripts/evaluate_policy.py --episodes 100
.venv/bin/python scripts/render_policy.py --seed 64002
.venv/bin/python scripts/render_policy.py \
--mode untrained --seed 64002
```

The launcher loads saved weights; it does not retrain. Seed 64002 creates a matched visual example. Keep that example separate from the 100-episode aggregate.

15 Held-Out Results

On 100 unseen randomized episodes, the trained policy completed the entire task in 86%. Right grasp succeeded in 91%, handoff position and left transfer in 89%, left hold in 89%, and throw in 86%.

Table 9: Additional held-out metrics

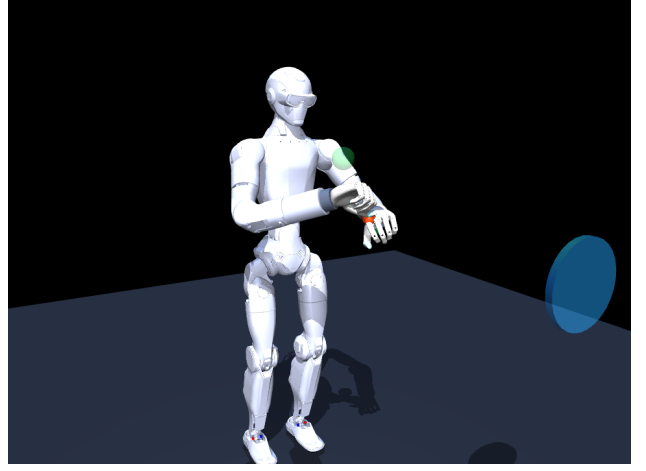
Metric	Mean value
Minimum right reach error	10.617 mm
Forward ball travel	0.202 m
Peak forward ball speed	1.356 m/s
Right-hand contacts	9.05
Left-hand contacts	7.17

For matched seed 64002, the trained rollout completed the task and traveled 0.278 m forward. The untrained network had zero success, 59.83 mm minimum reach error, and no hand-ball contacts.

16 Visual Result and Interpretation



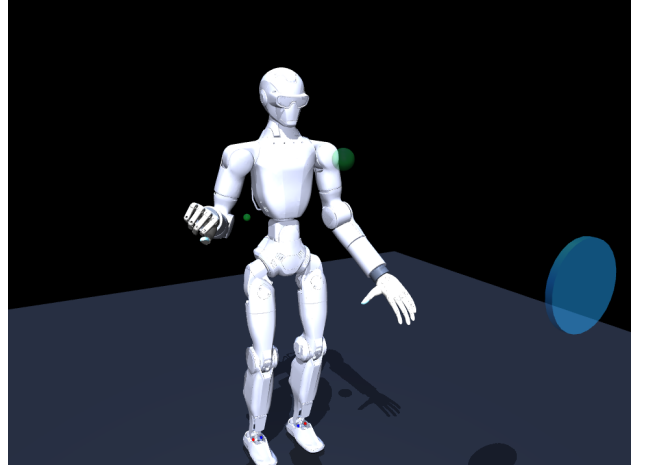
Right-hand grasp.



Ball received by the left hand.



Left-hand release and follow-through.



Untrained rollout for the matched task seed.

Figure 1: Trained task stages and the untrained comparison condition.

The stage rates localize most failures before or during the first grasp. Once the ball reaches the left hand, transfer and throw rates are comparatively close. More randomized right-hand approach data and improved adapter calibration are the next logical simulation studies.

17 Final Checklist and References

- ☐ Both stock terminal hand geometries are absent from the R1 XML.
- ☐ Both Revo 2 models expose six controls and correct mesh paths.
- ☐ The policy reads 94 observations and produces 22 actions at 40 Hz.
- ☐ Training records 140 teacher episodes, two DAgger rounds, and 55,926 samples.
- ☐ The contact-gated stabilizer is disclosed in reported results.
- ☐ Evaluation uses held-out seeds and reports each task stage.
- ☐ One hypothesis and one changed parameter are recorded before training.
- ☐ The edited code and command-line parameters are saved with the results.

References

1. BrainCo Hand. (n.d.). *Revo 2 parameters*.
<https://www.brainco-hz.com/docs/revolimb-hand-dev/en/revo2/parameters.html>
2. DeepMind. (2026). *MuJoCo computation and simulation documentation*.
<https://mujoco.readthedocs.io/en/stable/computation/index.html>
3. Kingma, D. P., and Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations*.
<https://arxiv.org/abs/1412.6980>
4. Ross, S., Gordon, G., and Bagnell, D. (2011). A reduction of imitation learning and structured prediction to no-regret online learning. *Proceedings of AISTATS*, 15, 627–635.
<https://proceedings.mlr.press/v15/ross11a.html>
5. Unitree Robotics. (n.d.). *R1 humanoid robot*.
<https://www.unitree.com/R1>